# Week 49

**v1.0.0**

## Table of contents

1 Difference between $Q$ and $K_{eq}$ ..... 1
2 Determination of ΔH° for a reaction from the van't Hoff plot. ..... 5
3 ATP hydrolysis ..... 8

```
try:
    import fysisk_biokemi
    print("Already installed")
except ImportError:
    %pip install -q "fysisk_biokemi[pyodide] @ https://au-mbg.github.io/fysisk-biokemi/
wheels/fysisk_biokemi-0.2.2-py3-none-any.whl"
```

---

## 1 Difference between $Q$ and $K_{eq}$

```
import matplotlib.pyplot as plt
import pandas as pd
from IPython.display import display
pd.set_option('display.max_rows', 6)
```

Consider the reaction:

$$A \rightleftarrows B$$

The time course of the reaction was followed monitoring formation of the product $B$. The starting concentration of $A$ was 50 µM, and as you will learn later in the course, the reversible reaction going from $A$ to $B$ follows first order kinetics.

**(a) Load & convert**

The data is contained in the `diff_q_keq_data.xlsx` file.

```
from fysisk_biokemi.datasets import load_dataset
df = load_dataset('diff_q_keq') # Load from package for the solution so it doesn't require
to interact.
display(df)
```

| | Time_(min) | [B]_(uM) |
|---|---|---|
| 0 | 0 | 0.000000 |
| 1 | 1 | 12.756091 |

| | Time_(min) | [B]_(uM) |
|---|---|---|
| 2 | 2 | 21.896230 |
| ... | ... | ... |
| 33 | 33 | 44.999248 |
| 34 | 34 | 44.999461 |
| 35 | 35 | 44.999614 |

Convert the measured concentrations of $B$, $[B]$, to concentrations in the SI-unit M (mol/L). Repeat for the starting concentration of $A$, $[A_0]$.

```
## Your task: Convert the concentration of B to M and set in new column ##
df['[B]_(M)'] = df['[B]_(uM)'] * 10**(-6) # Unit: M

## Your task: Set the initial concentration of A, in the variable A0.
## Remember to convert to M.
A0 = 50 * 10**(-6) # Unit: M
```

**(b) Law of mass balance**

Use the laws of mass balance to calculate $[A]$ as a function of time in the unit M, add it as a column to the `DataFrame`

```
df['[A]_(M)'] = A0 - df['[B]_(M)']
```

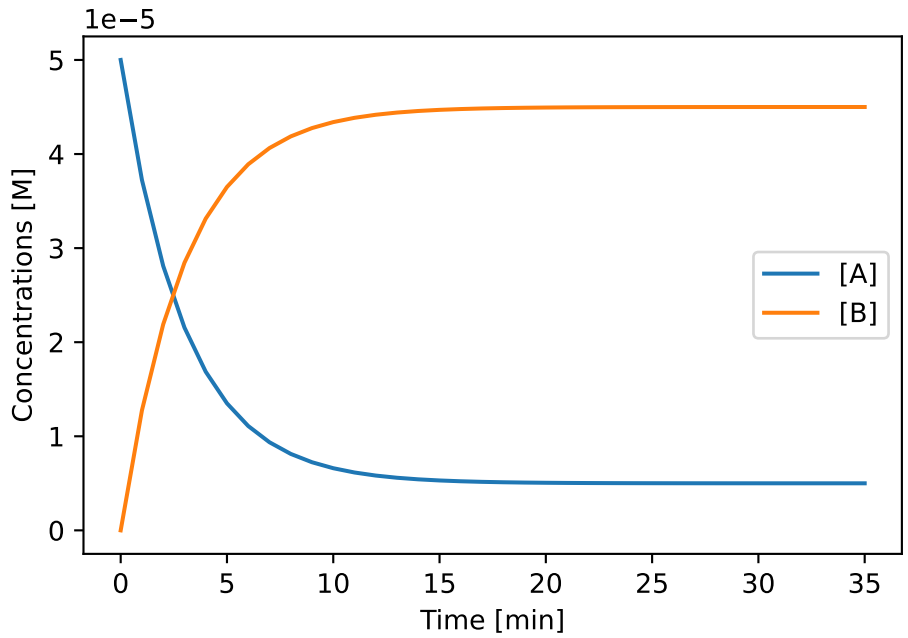
Make a plot of the time vs. the concentration of $A$ and of time versus the concentration of $B$.

```
fig, ax = plt.subplots()
## Your task: Plot time vs. concentrations.
ax.plot(df['Time_(min)'], df['[A]_(M)'], label='[A]') # Time vs. concentration of A
ax.plot(df['Time_(min)'], df['[B]_(M)'], label='[B]') # Time vs. concentration of B

## This shows the labels.
ax.legend()

## EXTRA: You do not need to understand this:
ax.set_xlabel('Time [min]')
ax.set_ylabel('Concentrations [M]')
```

```
Text(0, 0.5, 'Concentrations [M]')
```




**(c) Mass action ratio**

Calculate and plot the mass action ratio $Q$ as a function of time.

```
## Your task: Calculate the mass action ratio
Q = df['[B]_(M)'] / df['[A]_(M)']
```

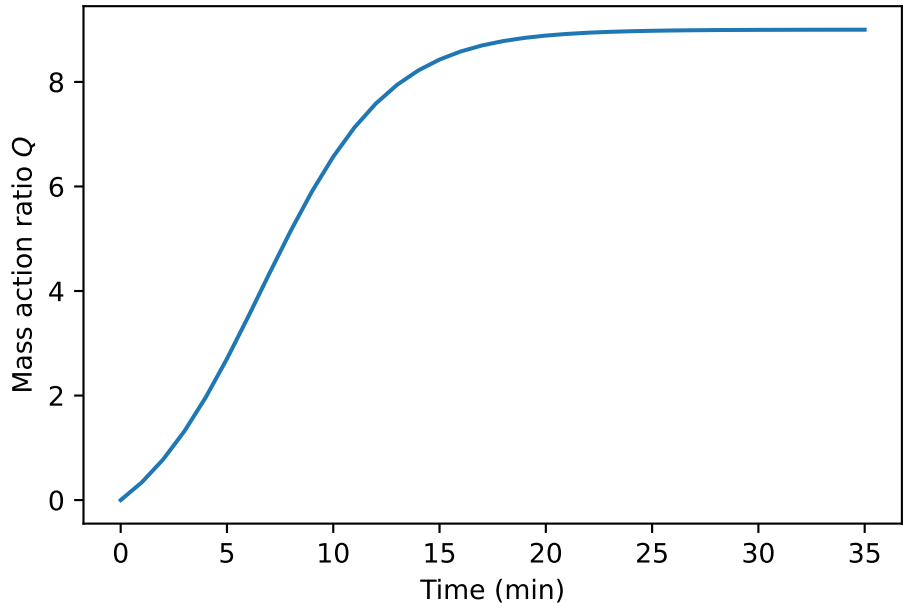
Now plot the mass action ratio versus time.

```
fig, ax = plt.subplots()

## Your task: Make a plot of the axis from the dataframe df['Time_(min)'] vs Q
ax.plot(df['Time_(min)'], Q)

## EXTRA: You do not need to understand this
## Adds customization.
ax.set_xlabel('Time (min)')
ax.set_ylabel('Mass action ratio $Q$')
```

```
Text(0, 0.5, 'Mass action ratio $Q$')
```




**(d) $Q$ vs $K_{eq}$**

What is the difference between $Q$ and $K_{eq}$?

```
"""
Q is the mass action ratio at any time throughout the reaction. Keq is the mass
action ratio at equilibrium only, and tells about the energetic favorisation of
one direction over the other.
"""
```

**(e) Favoured compound**

Which of the two compounds $A$ and $B$ is favored at equilibrium? (Favored: highest concentration)

```
"""
When the reaction reaches close to equilibrium, more B is present as compared to A, telling
that B is favored.
Keq > 1 --> B is favored
"""
```

**(f) Equilibrium constant**

What is the equilibrium constant of this reaction?

```
"""
Can also just read the concentrations values at eq from the graph and calculate K_eq from
those.
But it can of course also just be calculated.
"""
K_eq = df['[B]_(M)'][35] / df['[A]_(M)'][35]
print(K_eq)
```

```
8.999228315035122
```

---

## 2 Determination of ΔH° for a reaction from the van't Hoff plot.

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit
import numpy as np
from IPython.display import display
```

The equilibrium constant for a reversible reaction going from $A$ to $B$ was measured as a function of temperature.

The data obtained is given in the Excel document `deter_delta_h_data.xlsx`. Load the data with pandas.

```
from fysisk_biokemi.datasets import load_dataset
df = load_dataset('deter_delta_h') # Load from package for the solution so it doesn't
require to interact.
display(df)
```

| | T_(K) | Keq |
|---|---|---|
| 0 | 283 | 0.87 |
| 1 | 288 | 0.90 |
| 2 | 293 | 0.93 |
| 3 | 298 | 0.97 |
| 4 | 303 | 1.00 |
| 5 | 308 | 1.03 |

### (a) Transform & plot the data

> Tip
>
> In NumPy the `np.log` function calculates the natural logarithm that we often write as ln and `np.log10` calculates the base 10 logarithm.

Add new columns to the `DataFrame` containing $\ln(K_{eq})$ and $1/T$ and make a plot with $1/T$ on the x-axis and $\ln(K_{eq})$ on the y-axis.

```
## Your task: Add a column containing 1/T and one containing ln(Keq) ##
df['1/T'] = 1/df['T_(K)']
df['ln(Keq)'] = np.log(df['Keq'])
```

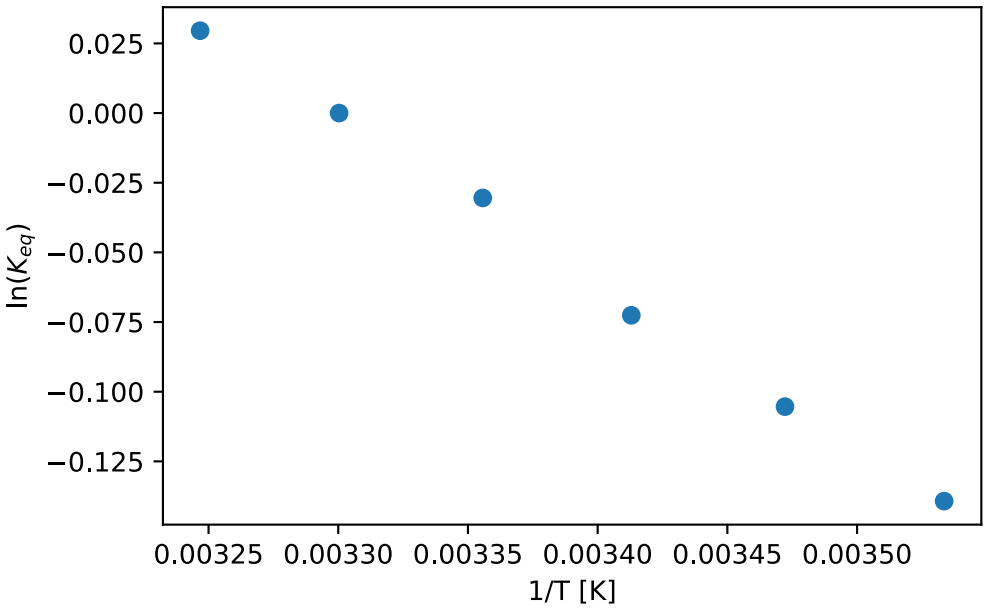
Now you can make the plot in the cell below

```
fig, ax = plt.subplots()

## Your task: Make a plot of 1/T vs ln(Keq)
ax.plot(df['1/T'], df['ln(Keq)'], 'o')

## EXTRA: Customization you don't need to edit or understand this
## It adds axis labels to the plot you made.
ax.set_xlabel(r'1/T [K]')
ax.set_ylabel(r'$\ln(K_{eq})$')
```

```
Text(0, 0.5, '$\\ln(K_{eq})$')
```




**(b) Fit trendline**

Make a linear trendline by fitting a linear function to the data.

Start by defining a linear function

```
def vant_hoff(T, a, b):
    ## Your task: Implement a linear function
    result = a * T + b
    return result
```

Now we can use regression to find the parameters

```
## Your task: Make a fit with curve_fit to find the parameters ##
fitted_parameters, trash = curve_fit(vant_hoff, df['1/T'], df['ln(Keq)'])
a_fit, b_fit = fitted_parameters

## Print ##
```

```
print(a_fit)
print(b_fit)
```

```
-599.0182388271635
1.975814074075527
```

Then make a plot with the trendline added

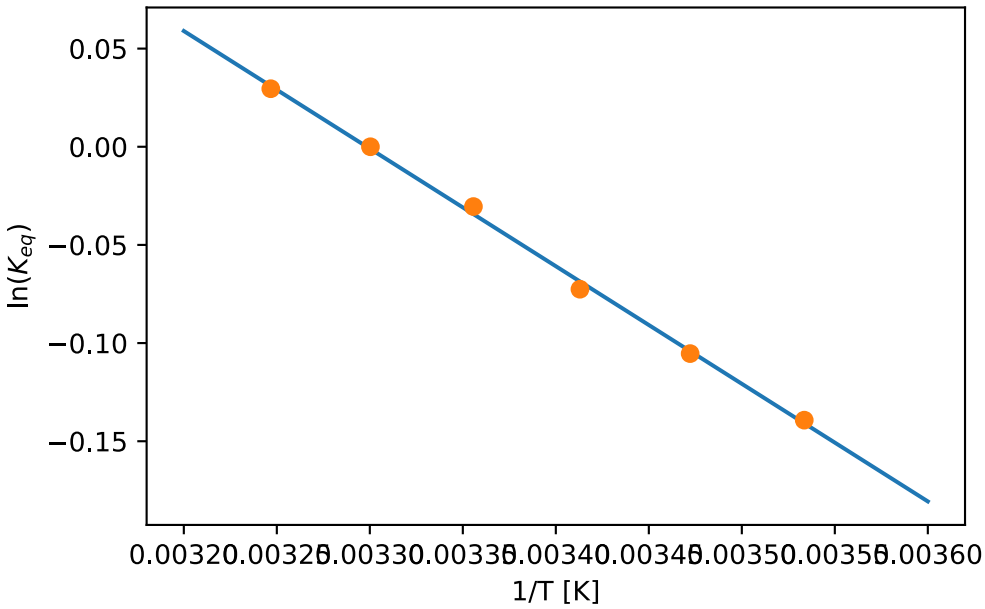
```
fig, ax = plt.subplots()

## Your task: Choose points to evaluate the fit
## Look at the x-axis above and estimate a value for the minimum and maximum.
## Then calculate the fit at those points
max_x = 0.0032
min_x = 0.0036
T_trend = np.array([min_x, max_x])
y_fit = vant_hoff(T_trend, a_fit, b_fit)

## Your task: Plot the fit
ax.plot(T_trend, y_fit)

## Plots the data
ax.plot(df['1/T'], df['ln(Keq)'], 'o')

## EXTRA: You do not need to edit or understand this.
## It adds axis labels to the plot you made.
ax.set_xlabel('1/T [K]')
ax.set_ylabel(r'$\ln(K_{eq})$');
```




**(c) Reaction type**

Calculate the reaction enthalpy and entropy, at $T = 298\,\mathrm{K}$, assuming that $R = 8.314472 \times 10^{-3}\ \frac{\mathrm{kJ}}{\mathrm{mol \cdot K}}$. Is the reaction endo- or exothermic?

```
## Your task: Set known values and add units as comments e.g. # kJ/(mol K) ##
R = 8.314472 * 10**(-3) # Unit: # kJ/(mol K)
T = 298 # Unit: K


## Your task: Calculate ΔH and ΔS ##
## Consider the units!
dH = -R * a_fit # kJ/mol
dS = R * b_fit # kJ/(mol K)

print(dH)
print(dS)
```

```
4.980520374217764
0.016427850796106896
```

> **Tip**
>
> The units of a slope is the units of the y-axis dvided by the units of the x-axis.
>
> In the van't Hoff plot the y-axis is unitless and the x-axis is in 1/K.

**(d) Free energy change**

Calculate the free energy change, $\Delta G^\circ$, at $T = 298\,\mathrm{K}$, assuming that $R = 8.314472 \times 10^{-3}\ \frac{\mathrm{kJ}}{\mathrm{mol \cdot K}}$

```
## Your task: Calculate ΔG
dG = dH - T * dS

print(dG)
```

```
0.08502083697790841
```

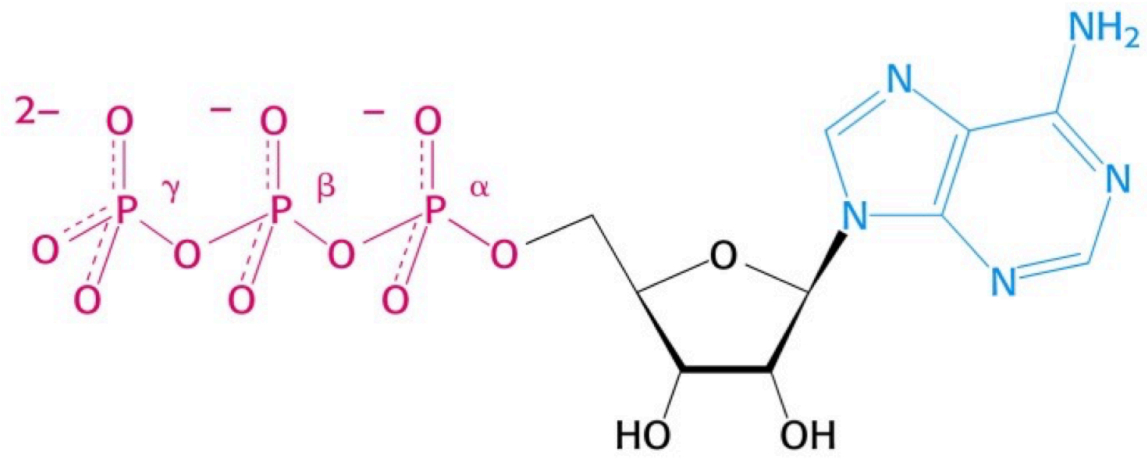
---

# 3 ATP hydrolysis

```
import numpy as np
```

Below is shown the structure of ATP.

$\Delta G^\circ$ for the hydrolysis of ATP to ADP and Pi is $-30.5\,\mathrm{kJ/mol}$.




**Adenosine triphosphate (ATP)**

### (a) $\Delta G$

In cells at 298 K the steady state concentrations of ATP, ADP og Pi resulted in a value for the reaction quotient $Q = \frac{[\text{ADP}][\text{Pi}]}{[\text{ATP}]} = 0.002$. Calculate ΔG' under these conditions.

```
## Your task: Define known values
## Add units as comments! E.g. # kJ / mol
Q = 0.002
T = 298 # K
dG_std = -30.5 # kJ / mol
R = 8.314 * 10**(-3) # kJ /(mol K)

## Your task: Calculate dG
dG = dG_std + R*T*np.log(Q)
print(dG)
```

```
-45.89713901562406
```

### (b) Equilibrium or not?

Assuming that the temperature within these cells is 298 K, are these cells in equilibrium with respect to the hydrolysis of ATP?

```
K_eq = np.exp(-dG_std/(R*T))
print(K_eq)
```

```
222001.499219259
```

```
"""
Q is very different from this value, and therefore these cells are very far from
equilibrium with respect to hydrolysis of ATP. This value of Q further favors the
hydrolysis of ATP
"""
```

### (c) ATP synthesized

Calculate the number of moles of ATP synthesized by a resting person in a period of 24 hours, assuming that the energy consumption by this person is 8368 kJ per 24 hours and that 40% of this energy is converted directly into ATP without loss of energy. Use the ΔG' value calculated in question a, and assume that 298 K.

```
# Your task: Set the known value of the energy spent for ATP.
energy_atp = 0.4 * 8368 # kJ

# Your task: Calculate the number of moles of ATP synthezied
synth_atp = energy_atp / np.absolute(dG)
print(synth_atp)
```

```
72.92829295657326
```

> Tip
>
> You can use `np.absolute` to calculate the absolute value of a number.

### (d) Turnover

Next, calculate the average number of times each ATP molecule is turned over (synthesis followed by hydrolysis) in 24 hours when the total amount of ATP in the body equals 0.2 mol.

```
## Your task: Set the known total_atp value
total_atp = 0.2 # Unit: mol

## Your task: Calculate the turnover number.
turn_over = synth_atp / total_atp # Unit mol/mol -> Unitless
print(turn_over)
```

```
364.64146478286625
```